

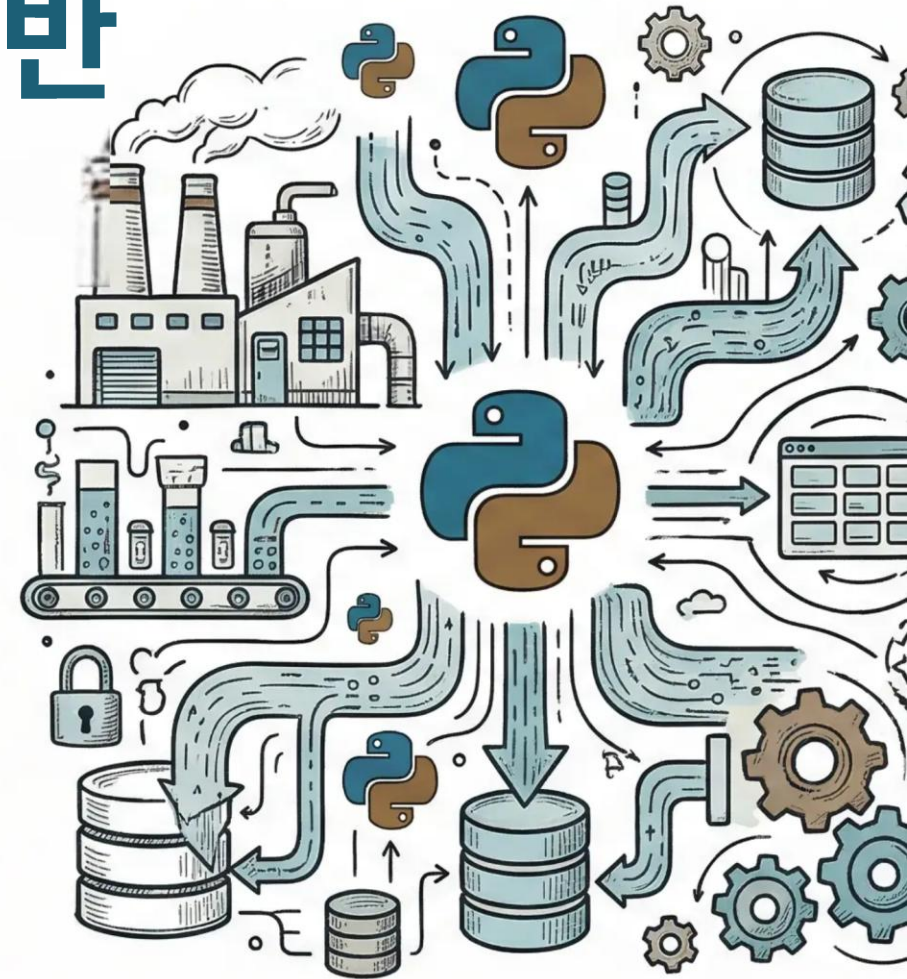
[기술공통] 데이터 엔지니어링



제조 AX를 위한 Python 기반 데이터 엔지니어링 실무

**pandas · 데이터 전처리 · 데이터 수집 ·
SQLite 적재 · ETL · 품질 점검**

3주간의 실무 중심 과정으로 제조 현장의 데이터를
Python으로 처리하고 활용하는 방법을 학습합니다



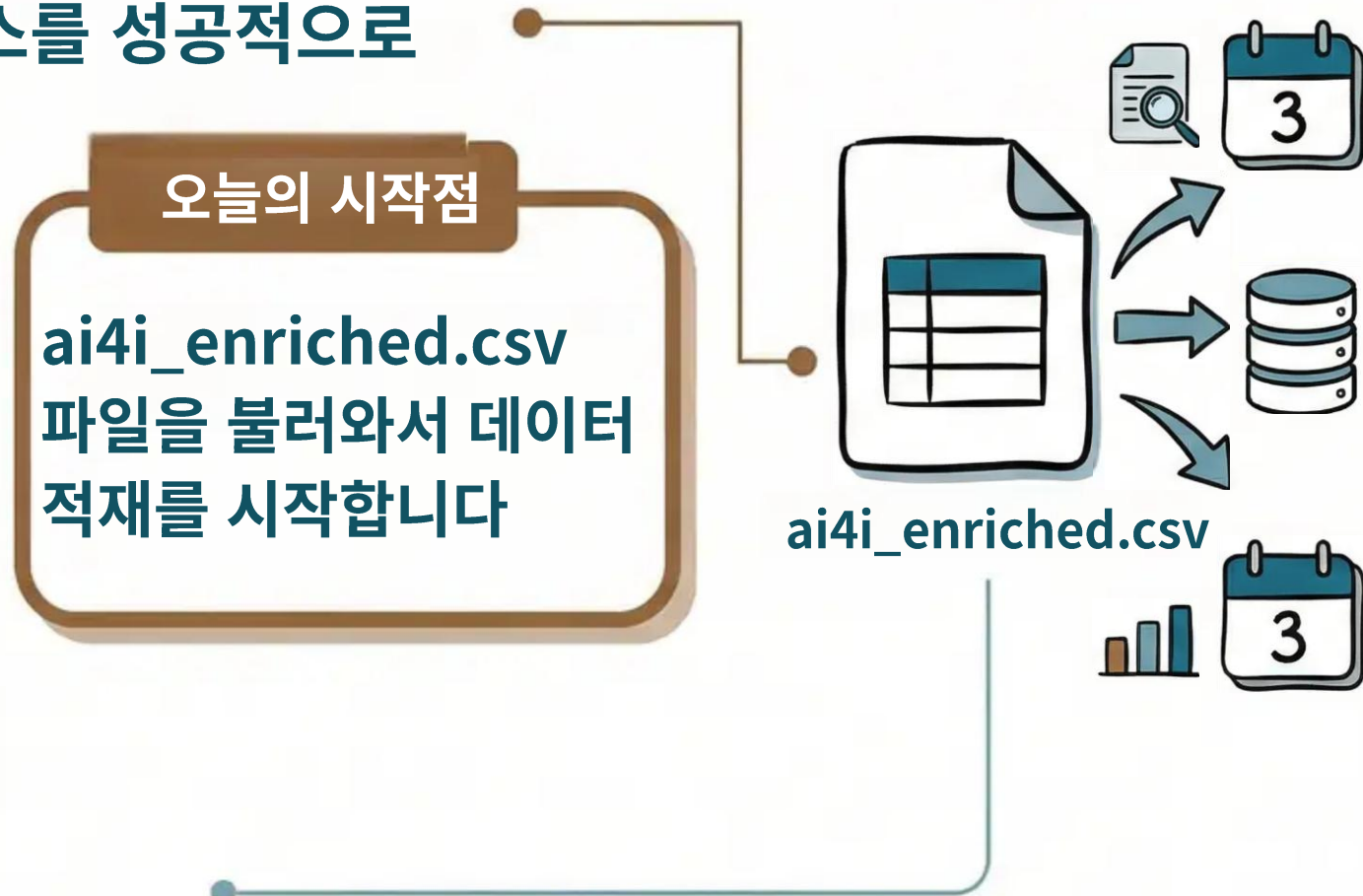
지난주 결과물 다시 보기

지난 2주차에서 여러 데이터 소스를 성공적으로 통합했습니다

ai4i_enriched.csv 파일이 생성되었습니다

1주차 정제 데이터 + 기준정보 + 여러 CSV 파일 + JSON 데이터

이 통합 파일을 SQLite 데이터베이스에 적재할 예정입니다



전체 3주 흐름 다시 보기

데이터 엔지니어링의 완성된 여정

1

1주차: 정제 (Clean)

- 원본 데이터 이해와 품질 점검
- pandas 기반 전처리 수행
- pandas 기반 전처리 수행
- ai4i_cleaned.csv 생성

2

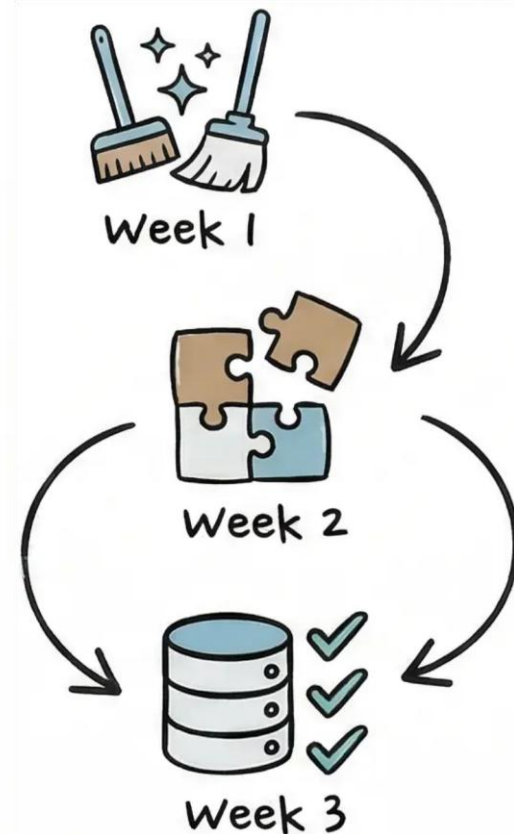
2주차: 통합 (Integrate)

- 다양한 데이터 소스 결합
- 파일, JSON, API 데이터 처리
- 파일, JSON, API 데이터 처리
- ai4i_enriched.csv 완성

3

3주차: 적재와 검증 (Load & Validate)

- SQLite 데이터베이스 적재
- 품질 점검과 로그 관리
- 품질 점검과 로그 관리
- 완전한 ETL 파이프라인 구축



오늘은 데이터 엔지니어링 여정의 마지막 단계입니다.
정제하고 통합한 데이터를 안전하게 저장하고 검증하는 방법을 배웁니다.

3주차 학습 목표



• SQLite 데이터베이스 연결

- Python에서 SQLite 연결 방법 학습
- 데이터베이스 파일 생성 및 관리



• 데이터 적재 (Load)

- DataFrame을 SQLite 테이블로 저장
- 적재 전략과 옵션 설정



• SQL 쿼리 실행

- 기본 SELECT 문으로 데이터 조회
- 조건 검색과 집계 쿼리 활용



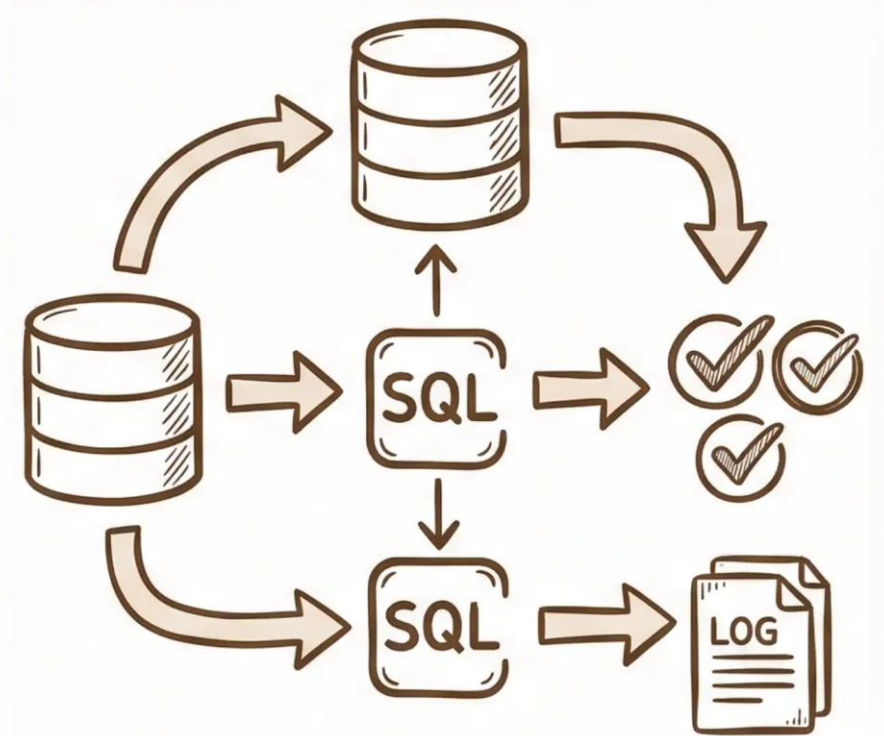
• 데이터 품질 점검

- 적재 후 데이터 검증 방법
- 결측치, 중복, 범위 오류 확인



• ETL 로그 관리

- 실행 기록 저장 및 추적
- 품질 점검 리포트 생성



오늘 만들 최종 결과물

실무 환경을 모사한 구체적인 산출물



manufacturing_pipeline.db

SQLite 데이터베이스 파일로 정제된 제조 데이터 저장



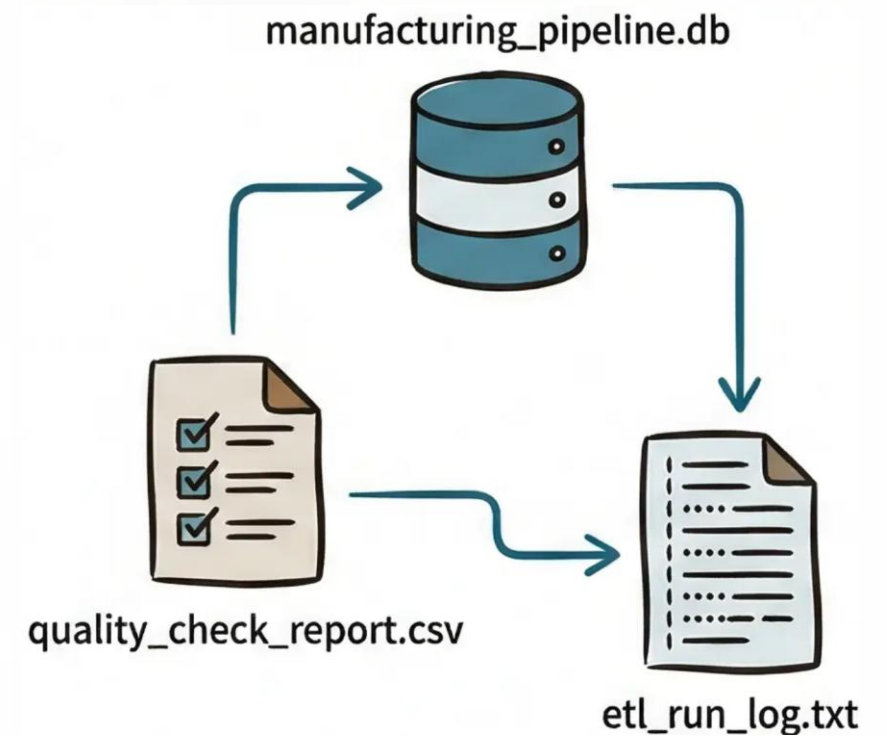
quality_check_report.csv

데이터 품질 점검 결과를 체계적으로 기록한 리포트



etl_run_log.txt

ETL 실행 과정과 결과를 추적 가능하게 기록한 로그



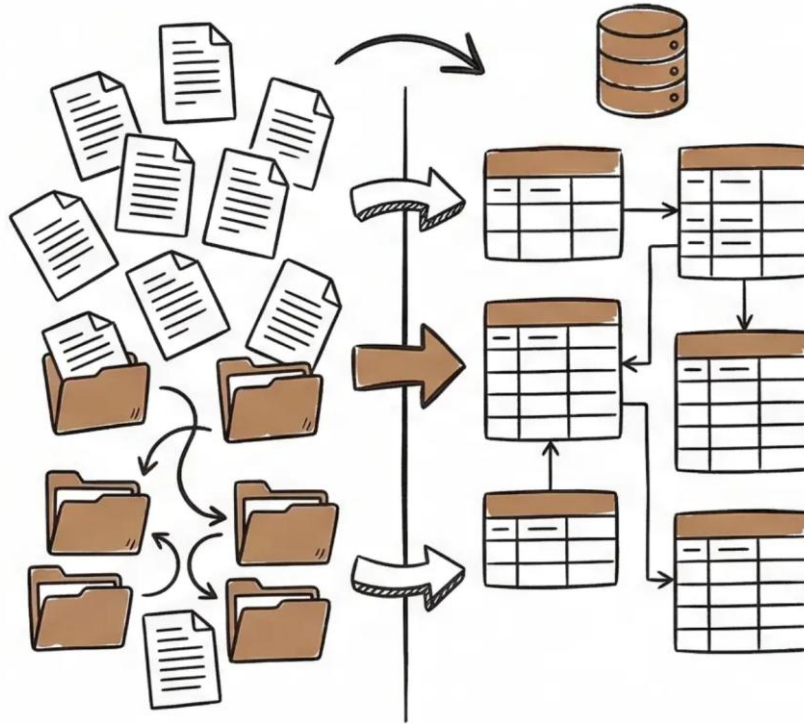
단순한 학습이 아닌 실제 현장에서 사용할 수 있는 형태의 결과물을 만들어봅니다

데이터베이스는 왜 필요한가

파일 저장의 한계



- 파일이 많아질수록 관리가 어려워짐
- 중복 데이터 발생 위험
- 동시 접근 시 충돌 문제
- 데이터 일관성 유지 어려움



데이터베이스의 장점



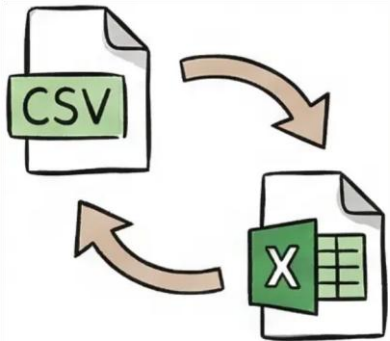
- 체계적인 구조로 데이터 관리
- 빠른 검색과 필터링
- 동시 접근 및 수정 지원
- 데이터 무결성 보장
- 백업과 복구 시스템

제조 현장에서의 실제 예시

생산 로그 파일 수백 개 → 통합 데이터베이스로 실시간 품질 모니터링

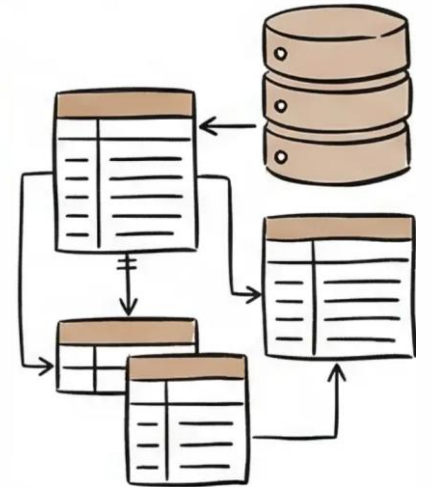
파일 저장과 DB 저장의 차이

파일 저장



- 단순하고 빠른 처리
- CSV, Excel 등 직관적 형태
- 즉시 읽고 쓰기 가능
- 개별 파일 단위 관리

DB 저장



- 구조화된 데이터 관리
- 관계와 제약조건 설정
- 동시 접근과 트랜잭션
- 체계적인 조회와 집계

역할이 다르므로 상황에 맞는 선택이 중요합니다

테이블, 행, 열 이해하기

- 테이블은 데이터베이스의 기본 저장 단위

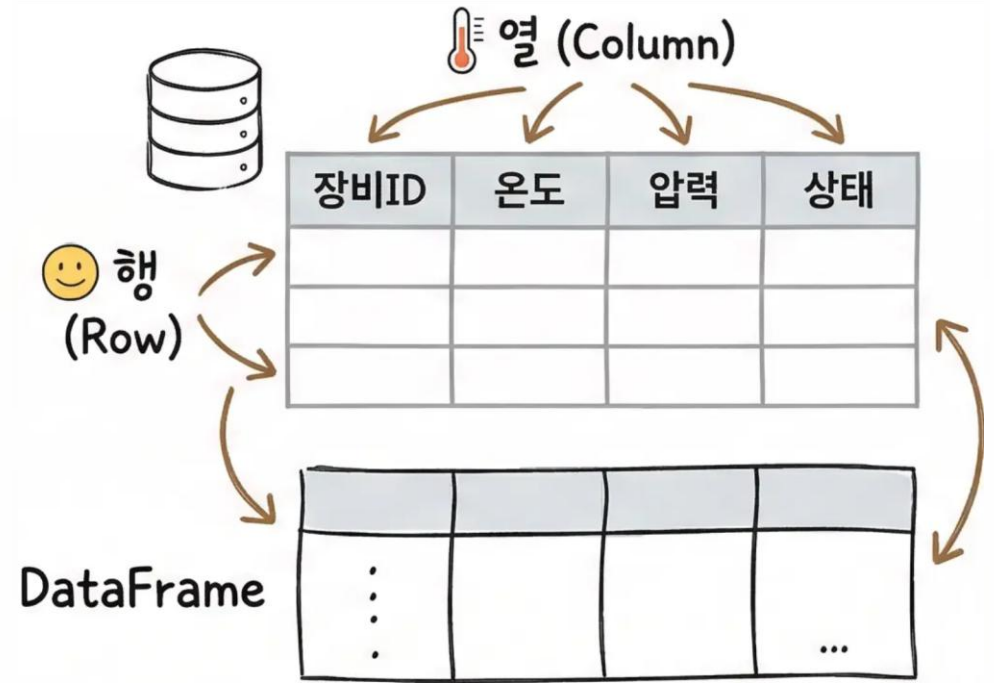
- 행(Row)과 열(Column)로 구성된 2차원 구조
- DataFrame과 매우 유사한 표 형태

- 행(Row) - 하나의 완전한 기록

- 한 개체에 대한 모든 정보
- 예: 한 대의 장비에 대한 전체 데이터

- 열(Column) - 특정 속성 정보

- 모든 기록이 공통으로 가지는 속성
- 예: 온도, 압력, 상태 등



DataFrame vs 테이블

- 구조: 둘 다 행과 열의 표 형태
- 차이점: 관리 방식과 저장 위치
- 장점: 테이블은 연구 저장과 다중 접근에 유리

기본 키는 왜 필요한가

한 행을 구분하는 기준

각 데이터 레코드를 고유하게 식별하는 컬럼

중복 방지의 핵심

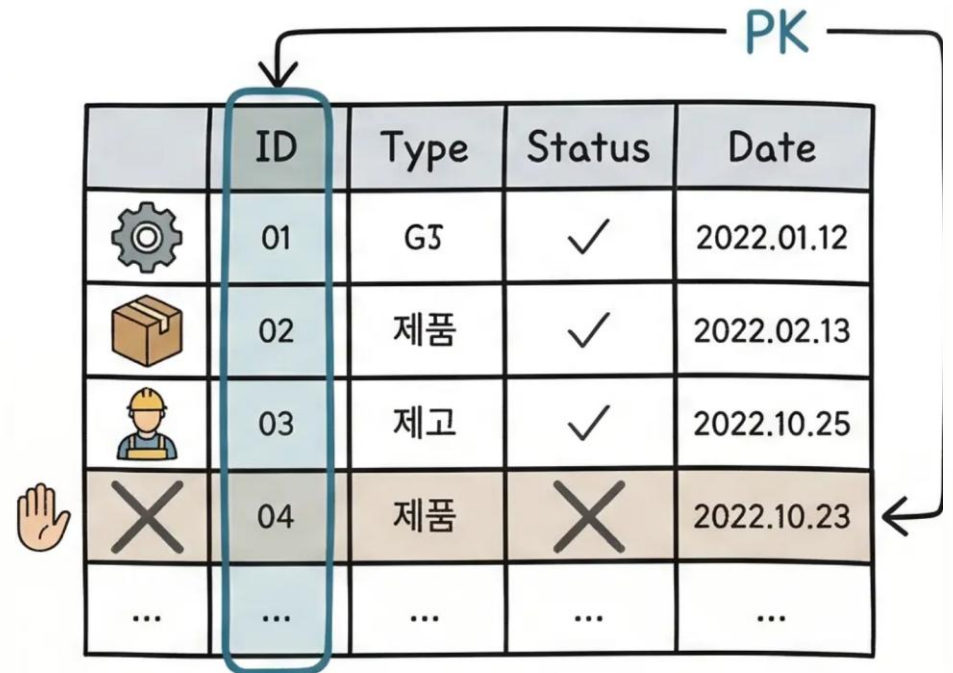
동일한 데이터가 여러 번 저장되는 것을 방지

데이터 추적성 확보

특정 기록을 정확하게 찾고 관리할 수 있음

실무 DB 설계의 기본

안정적인 데이터 관리를 위한 필수 요소



The diagram shows a database table with 5 columns: ID, Type, Status, and Date. The 'ID' column is highlighted with a blue box and a blue arrow pointing to it from the label 'PK' (Primary Key). A hand icon is pointing to the 'ID' column. The table contains 5 rows of data. The first row has a gear icon, ID 01, Type G3, Status checkmark, and Date 2022.01.12. The second row has a box icon, ID 02, Type 제품, Status checkmark, and Date 2022.02.13. The third row has a worker icon, ID 03, Type 제고, Status checkmark, and Date 2022.10.25. The fourth row has a red X icon, ID 04, Type 제품, Status red X, and Date 2022.10.23. The fifth row has ellipses in all columns. A curved arrow points from the 'ID' column to the 'Date' column, indicating a relationship.

	ID	Type	Status	Date
⚙️	01	G3	✓	2022.01.12
📦	02	제품	✓	2022.02.13
👷	03	제고	✓	2022.10.25
❌	04	제품	❌	2022.10.23
...	...	...	...	...

제조 현장 예시

설비 ID, 제품 번호, 작업 지시서 번호 등이 기본 키 역할을 수행하여 각 기록을 고유하게 식별

관계형 데이터베이스란 무엇인가

여러 테이블을 관계로 연결하는 구조

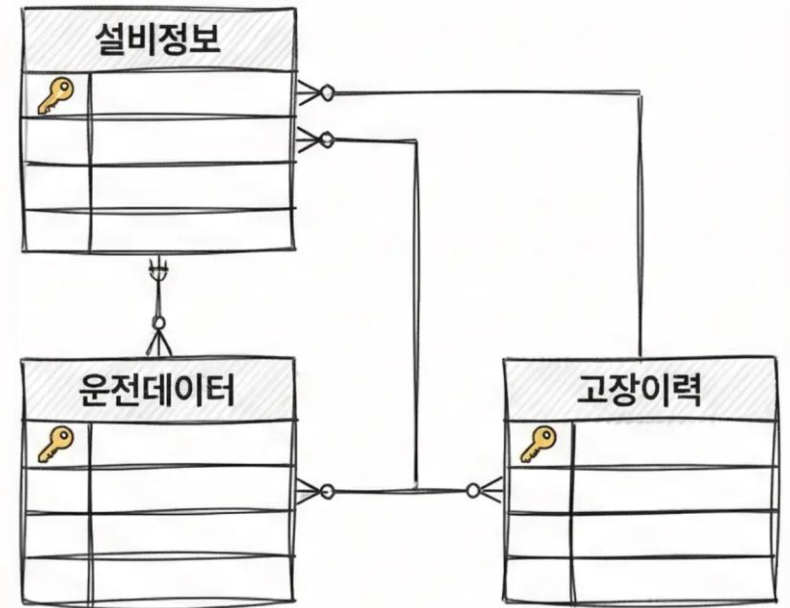
각 테이블은 특정 주제의 데이터를 저장
공통 키를 통해 테이블 간 연결

데이터 중복을 최소화

같은 정보를 여러 곳에 저장하지 않음
한 곳에서 수정하면 전체에 반영

데이터 무결성 보장

관계 규칙을 통해 데이터 품질 유지
잘못된 연결이나 삭제 방지

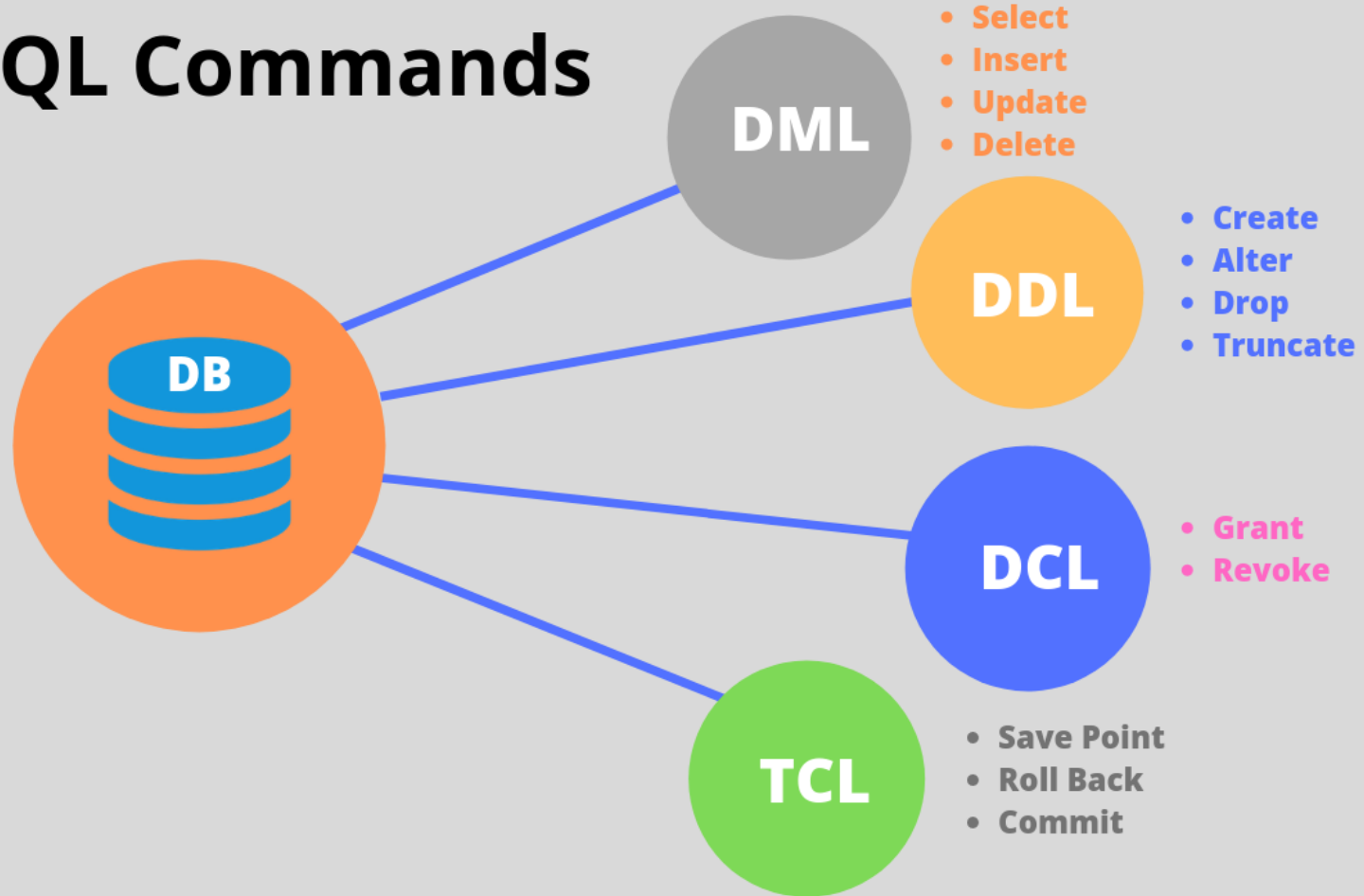


오늘은 한 테이블 위주로 실습

개념은 알고 가지만, 실습에서는 단일
단일 테이블로 ETL 흐름에 집중합니다

SQL이란?

SQL Commands



SQL이란 무엇인가

• 데이터베이스 표준 언어

- 관계형 데이터베이스를 다루는 국제 표준
- 거의 모든 데이터베이스 시스템에서 사용

• 핵심 4가지 작업 (CRUD)

- 조회 (SELECT) - 데이터를 읽어오기 
- 추가 (INSERT) - 새로운 데이터 삽입 
- 수정 (UPDATE) - 기존 데이터 변경 
- 삭제 (DELETE) - 데이터 제거 

• 실무에서 가장 자주 사용하는 쿼리

- SELECT: 데이터 조회 및 분석 (90% 이상)
- WHERE: 조건부 필터링
- GROUP BY: 그룹별 집계



Create



Read



Update



Delete

C

R

U

D



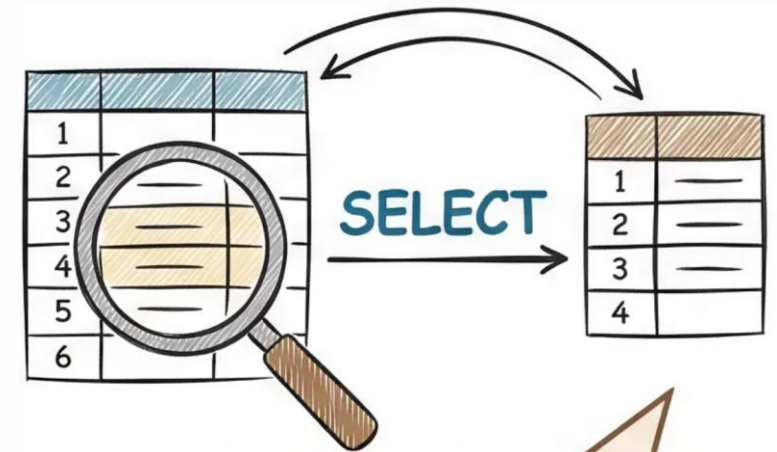
실무 TIP

제조 현장에서는 SELECT와 WHERE만 잘 써도 대부분의 데이터 분석이 가능합니다

SELECT로 데이터 조회하기

적재된 데이터를 확인하기 위한 가장 기본적인 SQL 조회 문법

- 기본 문법: **SELECT * FROM 테이블명**
- 전체 데이터 조회: 테이블의 모든 행과 열을 가져오기
- 특정 컬럼 선택: **SELECT 컬럼1, 컬럼2 FROM 테이블명**
- 샘플 데이터 확인: 적재 후 데이터 검증의 첫 단계



실무 팁

SELECT는 데이터를 읽기만 하므로 안전합니다
먼저 소량 데이터로 쿼리를 테스트해보세요

-- All data 조회

```
SELECT * FROM employee_table;
```

-- Specific columns 조회

```
SELECT employee_id, first_name, job_title FROM employee_table;
```

WHERE로 조건 검색하기

필요한 데이터만 골라내는 핵심 기능

• 필터링의 핵심 개념

- 전체 데이터에서 조건에 맞는 행만 선택
- pandas의 조건 필터링과 동일한 개념
- 데이터 분석의 첫 번째 단계

• 기본 WHERE 문법

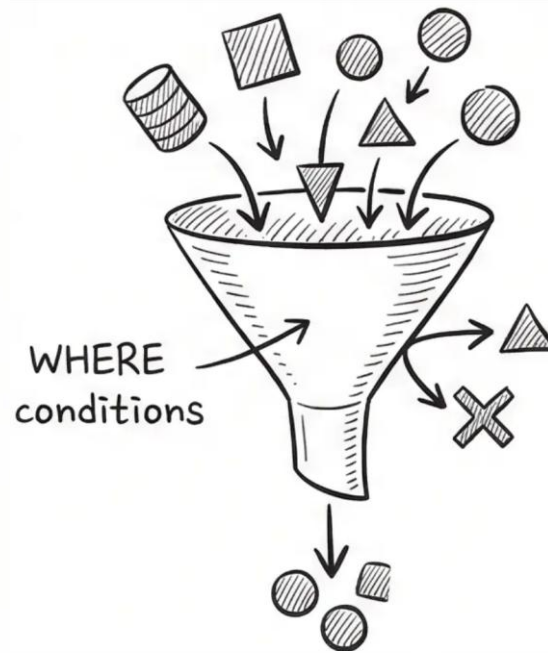
- `SELECT * FROM 테이블명 WHERE 조건`
- 숫자 비교: `WHERE 온도 > 25`
- 문자 비교: `WHERE 타입 = 'L'`
- 여러 조건: `WHERE 온도 > 25 AND 상태 = '정상'`

`SELECT * FROM 테이블명 WHERE 조건`

숫자 비교: `WHERE 온도 > 25`

문자 비교: `WHERE 타입 = 'L'`

여러 조건: `WHERE 온도 > 25 AND 상태 = '정상'`



실무 활용 예시

- 고장 데이터만 조회하기
- 특정 온도 범위 필터링
- 특정 기간의 데이터 추출
- 이상치 데이터 찾기

ORDER BY와 LIMIT

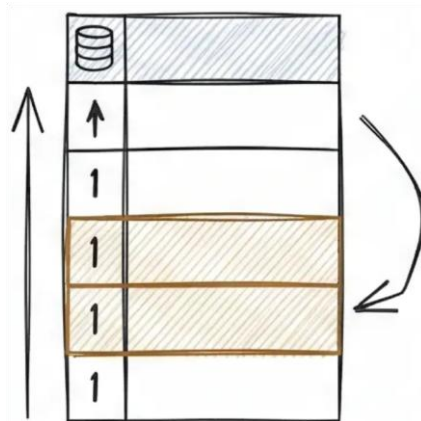
정렬과 일부 행 확인

ORDER BY로 정렬하기 하기

- 데이터를 원하는 순서로 정렬
- ASC (오름차순) / DESC (내림차순)
- 여러 컬럼 기준으로도 정렬 가능

LIMIT으로 샘플링

- 상위 N개 행만 조회
- 대용량 테이블에서 빠른 확인
- 페이징 처리의 기본



실무 활용 예시

- 최신 데이터 확인
- 상위/하위 값 찾기
- 샘플 데이터 추출
- 성능 개선된 조회

SQL 예시:

```
SELECT * FROM ai4i_data ORDER BY air_temperature DESC LIMIT 10;
```

GROUP BY로 집계하기

개념 설명:

SQL에서 특정 컬럼을 기준으로 데이터를 그룹화하고 집계 함수를 적용하는 기능

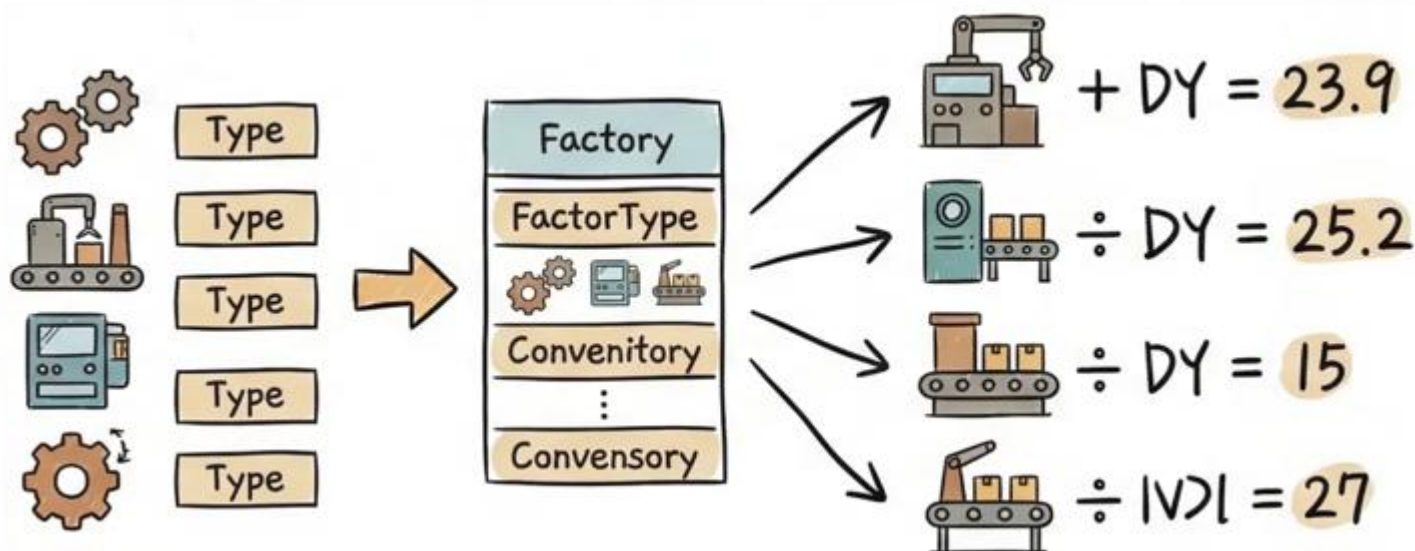
주요 집계 함수:

- COUNT(): 건수 계산
- AVG(): 평균 계산
- SUM(): 합계 계산
- MAX(), MIN(): 최대값, 최소값

```
SELECT Type,  
COUNT(*) as 장비수,  
AVG(Temperature) as 평균온도  
FROM ai4i_data  
GROUP BY Type
```

제조 현장 활용 예시

- 장비 유형별 평균 온도
- 교대별 생산량 집계
- 라인별 고장 건수
- 월별 품질 지표 요약



INSERT로 데이터 넣기

기본 INSERT 문법

- 테이블에 새로운 행 추가
- 값을 직접 지정하여 삽입
- 한 번에 하나 또는 여러 행 처리

예시 구문:

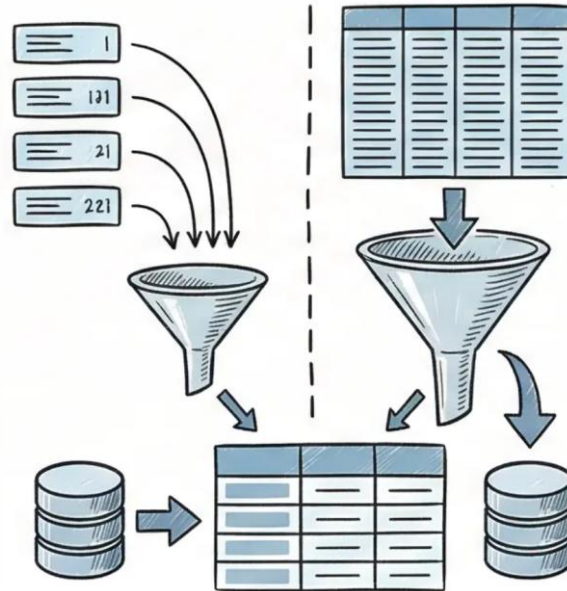
```
INSERT INTO table_name  
VALUES (값1, 값2, 값3);
```

pandas 대량 적재 방식

- to_sql()로 DataFrame 전체 삽입
- 수천 개 행을 한 번에 처리
- 실무에서 더 효율적인 방법

DataFrame → DB 적재:

```
df.to_sql('table_name',  
conn, if_exists='append')
```



실무 활용 포인트

- INSERT: 소량 데이터나 특정 조건
- to_sql(): 대량 데이터 일괄 처리
- 상황에 맞는 방식 선택이 중요

UPDATE로 데이터 수정하기

•UPDATE 구문의 기본 형태

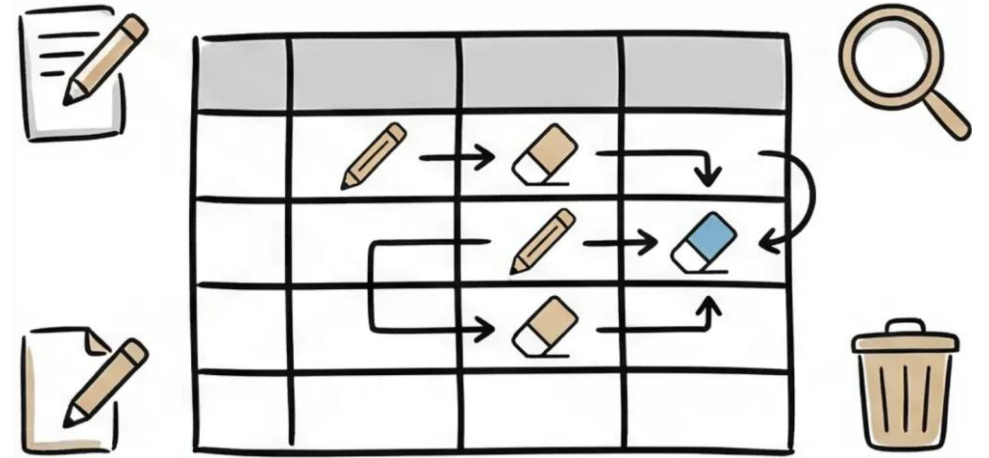
- 특정 조건에 맞는 데이터를 수정
- WHERE 절로 수정할 행을 정확히 지정

•언제 UPDATE를 사용하는가

- 잘못 입력된 데이터 수정
- 상태 정보 업데이트
- 계산된 값 갱신

•UPDATE 사용 시 주의사항

- WHERE 절 없으면 모든 행이 수정됨
- 실무에서는 매우 신중하게 사용
- 백업 후 실행하는 것이 안전



CRUD 개념 소개

- Create: 새 데이터 생성 (INSERT)
- Read: 데이터 조회 (SELECT)
- Update: 데이터 수정 (UPDATE)
- Delete: 데이터 삭제 (DELETE)

DELETE로 데이터 삭제하기

• 기본 DELETE 문법

- DELETE FROM 테이블명 WHERE 조건
- 조건 없이 실행하면 모든 데이터 삭제
- 되돌릴 수 없는 작업

• 실무에서 반드시 지켜야 할 원칙

- 삭제 전 반드시 백업 확인
- WHERE 조건을 먼저 SELECT로 확인
- 소량 테스트 후 본격 실행

• 제조 데이터 삭제 시나리오

- 테스트 데이터 정리
- 개인정보 보호 요구사항
- 보관 기간 만료 데이터



주의사항

실무에서는 DELETE보다 '삭제 플래그' 컬럼을 추가하여 논리적 삭제를 권장합니다.
물리적 삭제는 매우 신중하게 접근해야 합니다.

SQL을 배울 때 꼭 알아야 할 흐름



조회 (SELECT)

필요한 데이터를 테이블에서 가져오기
모든 SQL 작업의 시작점



조건 (WHERE)

특정 기준에 맞는 데이터만 선별
원하는 정보를 정확히 찾기



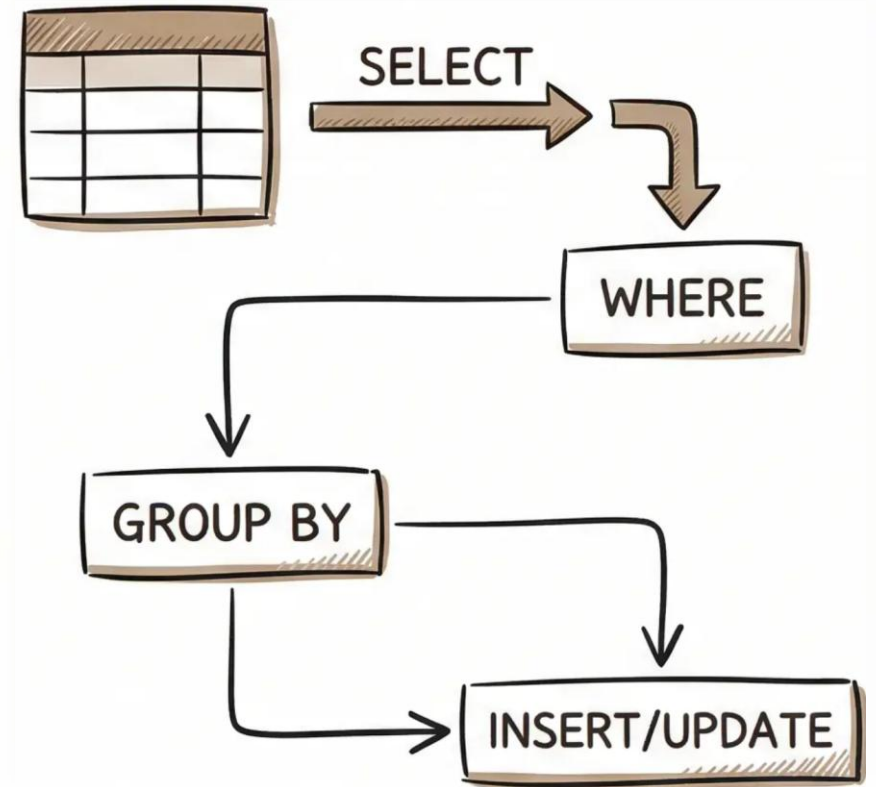
집계 (GROUP BY)

그룹별로 데이터를 요약하고 계산
의미 있는 통계와 인사이트 도출



저장/수정/삭제

데이터를 추가하거나 변경하기
데이터베이스 상태 관리



이 4단계만 익혀도 실무 SQL의 80%를 활용할 수 있습니다

왜 이번 강의는 SQLite를 쓰는가

설치와 설정이 필요 없음

별도 서버 구축 불필요

Python에 내장된 라이브러리 활용

파일 하나로 완결된 데이터베이스

.db 파일 = 전체 데이터베이스

이동과 백업이 간단함

학습 환경에 최적화

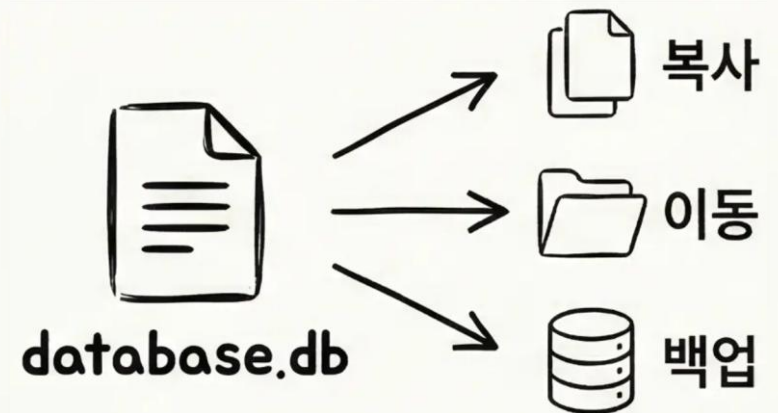
복잡한 설정 없이 바로 실습 가능

개념 이해에 집중할 수 있음



실무 팁

SQLite는 소규모 프로젝트와 프로토타이핑에 매우 적합합니다. 대, 소규모 운영 환경에서는 다른 데이터베이스를 고려해야 하지만, 데이터베이스 개념을 배우기에는 완벽한 선택입니다.



SQLite의 장점과 한계



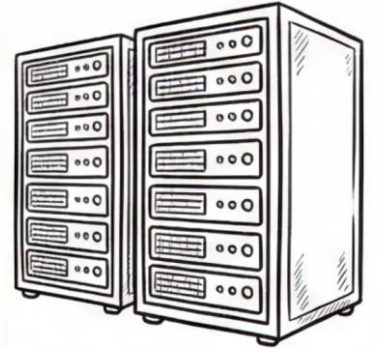
주요 장점



- 경량성: 별도 서버 설치 불필요
- 이식성: 단일 파일로 쉬운 이동과 백업
- 단순성: 복잡한 설정 없이 바로 사용
- 안정성: 검증된 오픈소스 데이터베이스
- Python 연동: 기본 라이브러리로 쉬운 연결



주요 한계



- 동시 접속: 다중 사용자 환경에 제한적
- 확장성: 대용량 데이터 처리에 한계
- 네트워크: 원격 접속 기능 부족
- 고급 기능: 복잡한 쿼리 최적화 제한
- 운영 관리: 엔터프라이즈급 관리 도구 부족



적절한 활용처

- 교육 및 프로토타이핑
- 로컬 데이터 분석
- 소규모 팀 프로젝트
- 임베디드 시스템

SQLite 파일 구조 이해하기

하나의 파일 = 하나의 데이터베이스

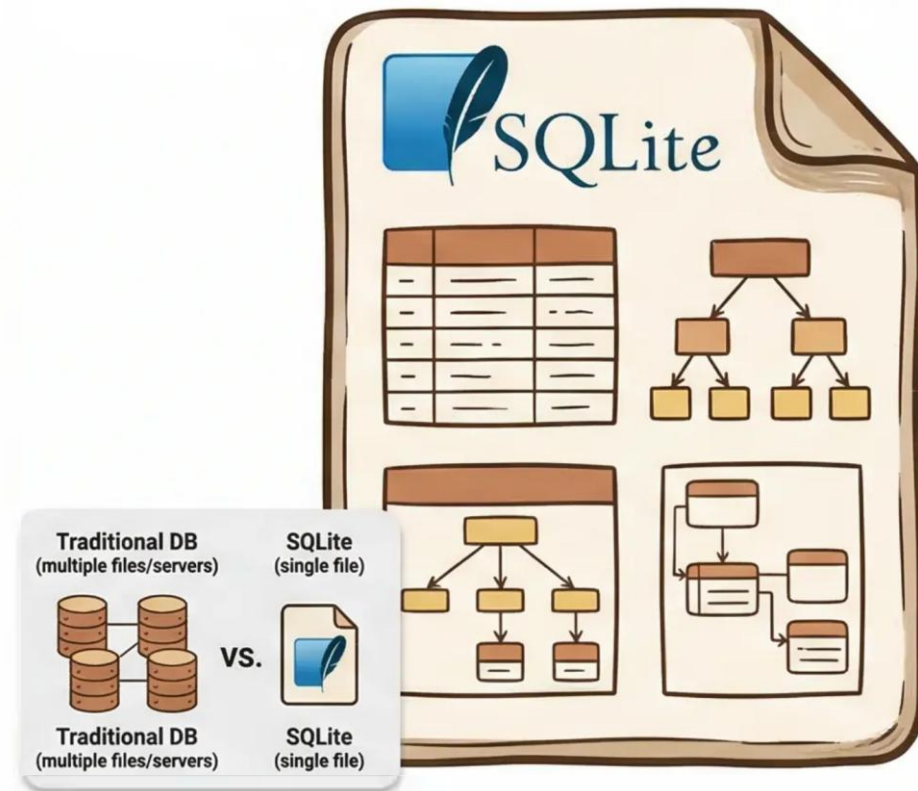
전체 DB가 .db 파일 하나로 저장됩니다
테이블, 인덱스, 스키마가 모두 포함됩니다
복잡한 설치나 서버 설정이 불필요합니다

파일 기반 관리의 장점

백업: 파일 하나만 복사하면 완료
이동: USB나 이메일로 쉽게 공유 가능
버전 관리: 날짜별 파일명으로 구분

실습에서의 활용

manufacturing_pipeline.db 파일 생성
결과물이 눈에 보여 학습에 효과적
프로젝트 폴더에서 직접 확인 가능



💡 실무 팁

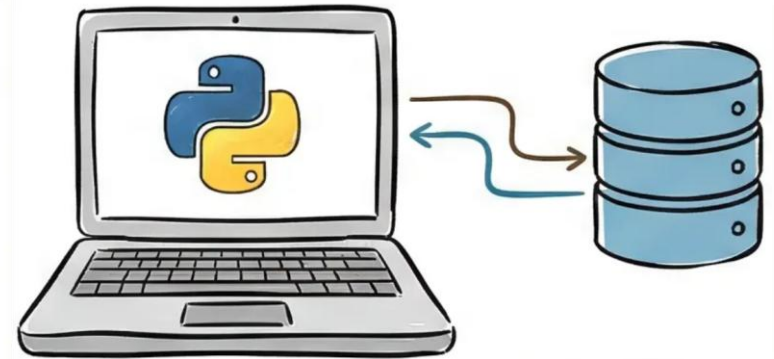
SQLite 파일은 프로젝트 결과물로 그대로 제출할 수 있어 보고서 작성성이 편리합니다!

Python에서 SQLite 연결하기

핵심 라이브러리: `sqlite3` 는 Python 표준 라이브러리로 별도 설치 불필요

연결 기본 구조:

`sqlite3.connect('파일명.db')` 로 간단한 연결



파일 자동 생성: 지정한 DB 파일이 없으면 자동으로 새로 생성

연결 객체 활용: `connection` 객체를 통해 이후 모든 DB 작업 수행

```
import sqlite3
```

```
# SQLite 데이터베이스 연결  
conn = sqlite3.connect('manufacturing.db')
```

```
# 작업 완료 후 연결 종료  
conn.close()
```


sqlite3.connect 사용하기

실제 DB 파일과의 연결을 시작하고 쿼리 실행을 위한 준비 단계를 실습합니다

• 연결 생성하기

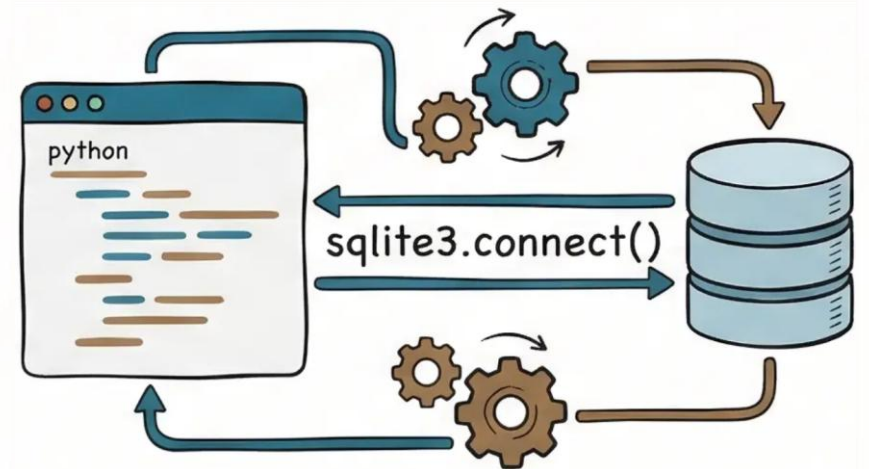
- `sqlite3.connect('파일명.db')` 함수 사용
- 파일이 없으면 자동으로 새 DB 생성

• 연결 객체 활용

- 쿼리 실행과 데이터 조작의 통로
- 여러 테이블 작업 시 하나의 연결로 처리

• 기본 사용 패턴

- 연결 → 작업 → 커밋 → 종료
- `with` 구문으로 안전한 연결 관리



💡 실무 팁

연결은 작업 시작 시 생성하고 완료 후 반드시 닫아야 합니다. 파일 경로가 틀리면 새 DB가 생성되니 주의하세요!

커서와 쿼리 실행 흐름

SQL 실행을 위한 통로와 처리 절차

• 커서란 무엇인가

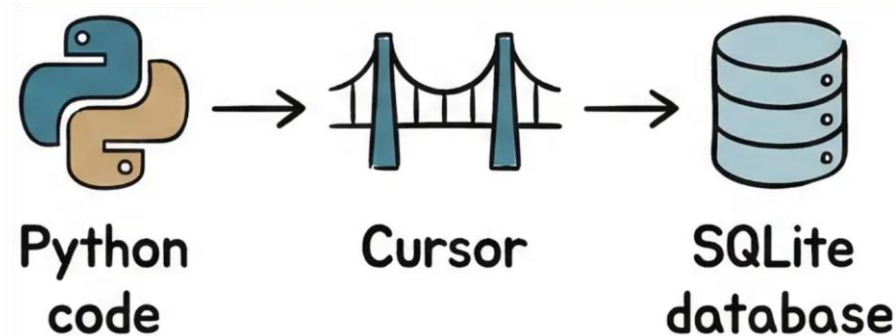
- SQL 명령을 데이터베이스에 전달하는 통로
- Python 코드와 SQLite 사이의 연결고리 역할

• 쿼리 실행의 기본 흐름

- 연결 생성 → 커서 생성 → SQL 실행 → 결과 처리
- 각 단계별 역할과 순서 이해

• pandas와 함께 사용할 때

- to_sql(), read_sql()이 내부적으로 커서 활용
- 복잡한 커서 조작 없이도 간편한 데이터 처리 가능



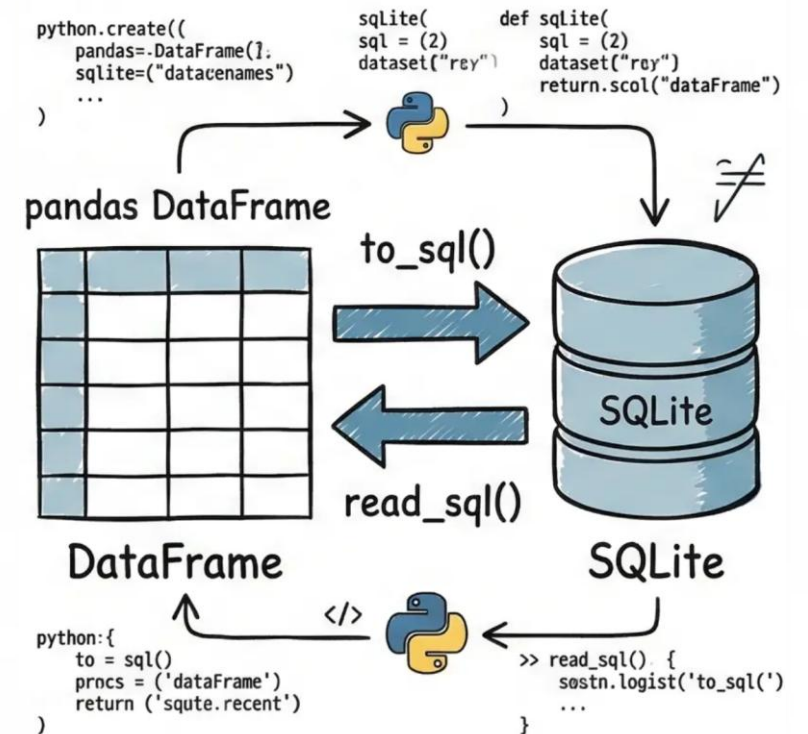
실습에서는 pandas 함수를 주로 사용하지만, 커서의 개념을 이해하면 데이터베이스 작업의 전체 흐름을 더 잘 파악할 수 있습니다

pandas와 SQLite 함께 쓰기

- Key Benefit: DataFrame과 DB를 쉽게 연결
- 복잡한 SQL 작성 없이 데이터 이동 가능
- Python 환경에서 일관된 작업 흐름 유지
- 핵심 장점들:
 - 간편한 직재: to_sql()로 DataFrame을 테이블로 변환
 - 쉬운 조회: read_sql()로 SQL 결과를 DataFrame으로 가져오기
 - 자동 변환: 데이터 타입 자동 매핑 지원

실습 효율성

pandas 연동으로 DB 작업이 DataFrame 조작만큼 쉬워집니다. 복잡한 데이터베이스 지식 없이도 실무 수준의 ETL 파이프라인을 구축할 수 있습니다.



to_sql과 read_sql 이해하기

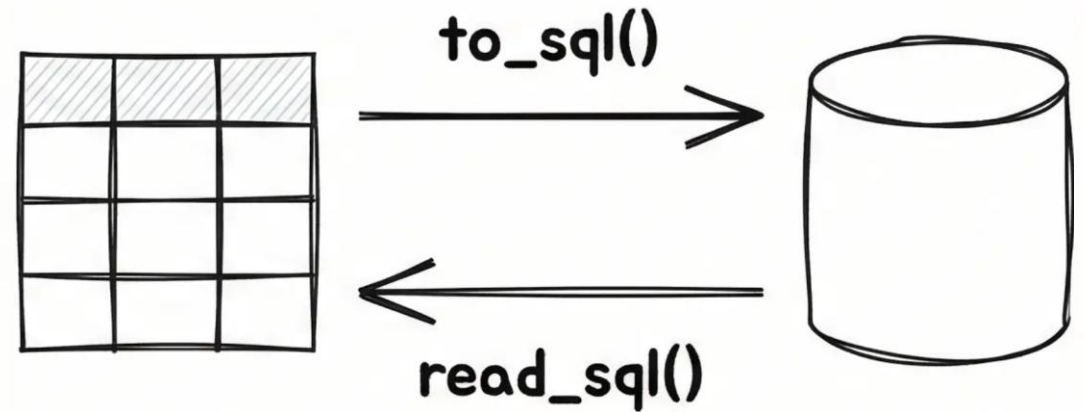
DataFrame과 SQLite 간 핵심 연동 함수

- **to_sql()** : DataFrame을 SQLite 테이블로 저장

- 테이블 자동 생성 가능
- 데이터 타입 자동 변환
- 기존 테이블 처리 옵션 제공

- **read_sql()** : SQLite에서 DataFrame으로 읽기

- SQL 쿼리 결과를 바로 DataFrame으로
- 조건부 데이터 추출 가능
- 검증과 분석에 필수적

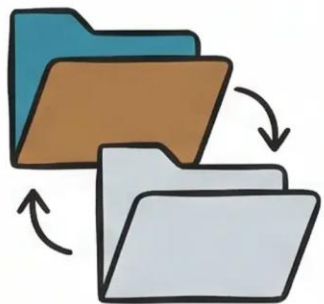


실무 팁

두 함수만 알면 파일과 DB 사이의
데이터 이동이 매우 간단해집니다

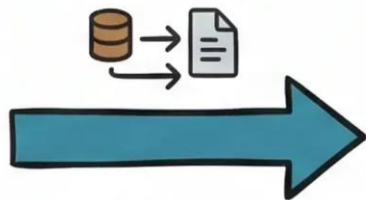
ETL 전체 흐름 다시 보기

Extract → Transform → Load의 완전한 데이터 파이프라인



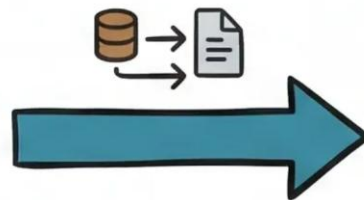
Extract (추출)

- 파일에서 데이터 읽기
- CSV, JSON, Excel 소스
- 2주차 결과물 활용



Transform (변환)

- 데이터 정제 및 검증
- 품질 점검 수행
- 필요한 형식으로 변환



Load (적재)

- SQLite 데이터베이스에 저장
- 테이블 생성 및 데이터 삽입
- 적재 후 검증 실행

3주간 학습한 모든 과정이 하나의 완성된 파이프라인으로 연결됩니다

Extract: 파일에서 데이터 읽기

ETL의 첫 번째 단계 - 통합 데이터 불러오기와 구조 점검

• 2주차 결과물 확인

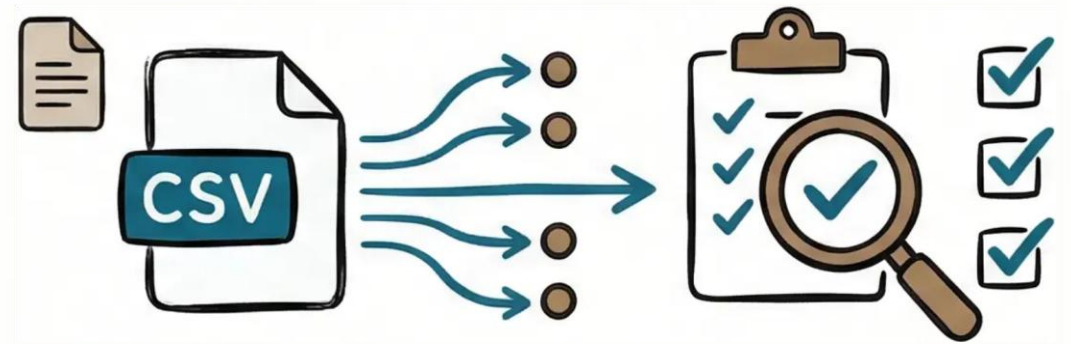
- ai4i_enriched.csv 파일 위치 확인
- 파일 크기와 수정 날짜 점검

• 데이터 읽기 실행

- `pandas.read_csv()`로 통합 파일 로드
- 인코딩과 구분자 설정 확인

• 구조 점검 필수 사항

- 전체 행과 열의 개수 확인
- `결(rng)`명과 데이터 타입 검증
- 주요 통계 정보 재확인



💡 적재 전 점검 포인트

- 예상 행 수와 일치하는가?
- 모든 필수 컬럼이 존재하는가?
- 데이터 타입이 올바른가?

Transform: 전처리 다시 적용하기

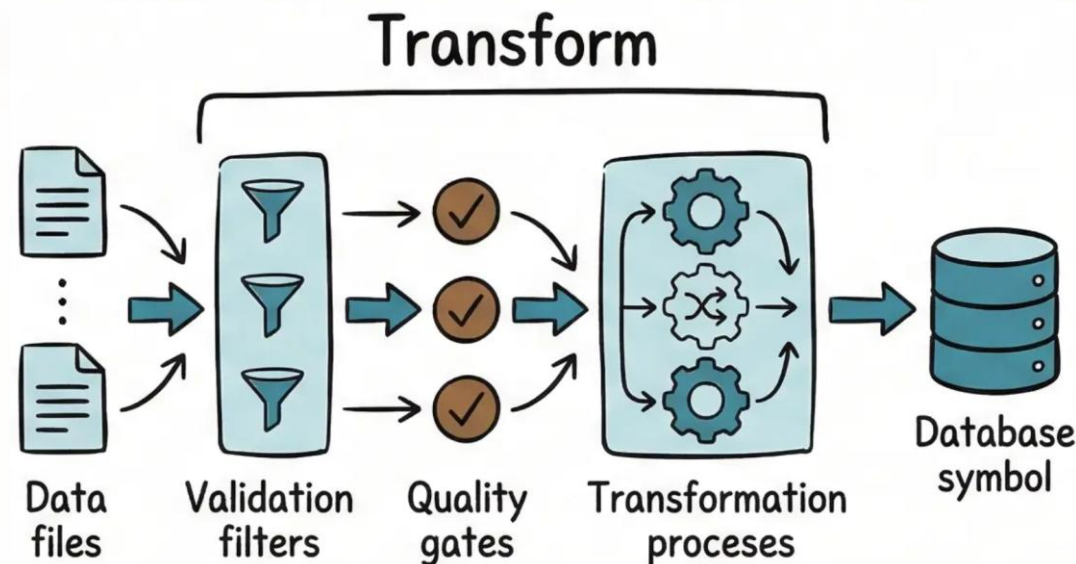
DB 적재 직전의 최종 품질 점검

• 적재 전 필수 점검 항목

- 🔍 결측치 및 중복 데이터 재확인
- 🔍 컬럼명과 자료형 최종 검증
- 🔍 값 범위 및 형식 오류 점검

• 최종 정제 작업

- 🔹 부적절한 데이터 필터링
- 🔹 컬럼 순서 및 이름 표준화
- ✅ 적재용 최적 형태로 변환

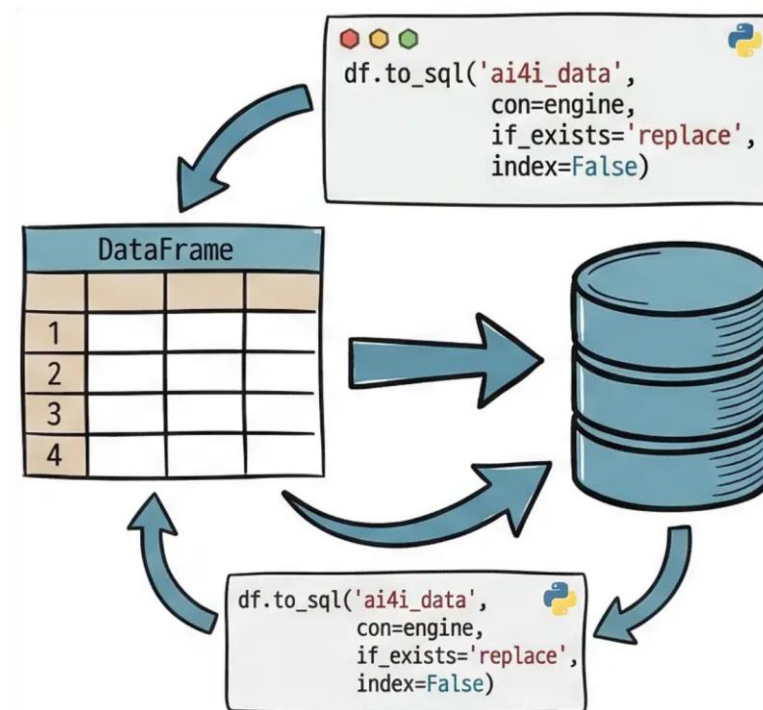


핵심 포인트

Transform은 단순 변환이 아닌
데이터 품질 보장의 마지막 관문입니다

Load: SQLite에 적재하기

- 정제된 DataFrame을 SQLite로 이동
- `to_sql()` 함수가 핵심 도구
- 테이블 생성과 데이터 삽입을 동시에 처리
- 기존 테이블 처리 방식 설정 가능
- 적재 옵션 이해하기
- `if_exists='replace'`: 기존 테이블 삭제 후 재생성
- `if_exists='append'`: 기존 데이터에 추가
- `index=False`: DataFrame 인덱스 제외하고 저장
- 적재 과정의 핵심 단계
- 연결 생성 → 데이터 전송 → 테이블 생성 → 연결 종료



실습 포인트

정제된 `ai4i_enriched.csv`를 `manufacturing_pipeline.db`의 `ai4i_data` 테이블로 적재하여 파일 기반 데이터를 구조화된 데이터베이스로 전환합니다



적재 전 테이블 설계 생각하기

- 데이터 타입 설정의 중요성
 - 숫자는 INTEGER, REAL로 명확히 구분
 - 문자열은 적절한 길이 제한 설정
 - 날짜/시간은 DATETIME 타입 활용
- 제약 조건 고려사항
 - PRIMARY KEY로 고유성 보장
 - NOT NULL로 필수 데이터 확보
 - CHECK 제약으로 값 범위 검증
- 실무 설계 원칙
 - 미래 확장성을 고려한 컬럼 설계
 - 인덱스 전략으로 조회 성능 최적화
 - 명명 규칙의 일관성 유지

DATABASE					
	COLUMN	TYPE	CL	C#	CONS.
🔑	id	INT	✓		
	hame	INTEGER	✓		NULL
	name	INTEGER	✓	×	
	came	VARCHAR	✓	×	
	email	VARCHAR	✓		
	email	INTEGER		×	CNLL
	connon_date	DATETIME(30)		×	
////					
	column	INT			
	account	DATA		✓	
	contact_name	VARCHAR(100)			



실습에서는 to_sql()로 단순화

개념 이해가 우선이므로 pandas의 기본 동작을 활용하고, 실무에서는 더 정교한 설계가 필요합니다



적재 후 데이터 검증하기

DB에 데이터가 의도대로 들어갔는지 샘플 조회와 통계 확인을 통해 검증합니다

- **샘플 데이터 조회로 기본 확인** ✓
 - SELECT 문으로 상위 몇 개 행 확인
 - 컬럼명과 데이터 타입이 예상과 일치하는지 점검
- **행 수 비교를 통한 데이터 무결성 검증** 🔍
 - 원본 DataFrame 행 수와 DB 테이블 행 수 비교
 - 데이터 유실이나 중복 적재 여부 확인
- **주요 통계값으로 데이터 품질 점검** 🔍
 - 수치형 컬럼의 최소/최대값, 평균값 확인
 - 결측치 개수와 고유값 개수 비교
- **검증 실패 시 대응 방안** 📈
 - 적재 과정 재실행 또는 오류 원인 분석
 - 로그 기록을 통한 문제 추적



row 수 비교로 적재 확인하기

- 핵심 개념: 데이터 적재 후 가장 기본적인 검증 방법
- 비교 대상: 원본 DataFrame의 행 수 vs DB 테이블의 행 수
- 검증 시점: 적재 직후 즉시 수행
- 판단 기준: 행 수가 일치하면 기본 검증 통과

필수 습관

적재할 때마다 반드시 확인하세요

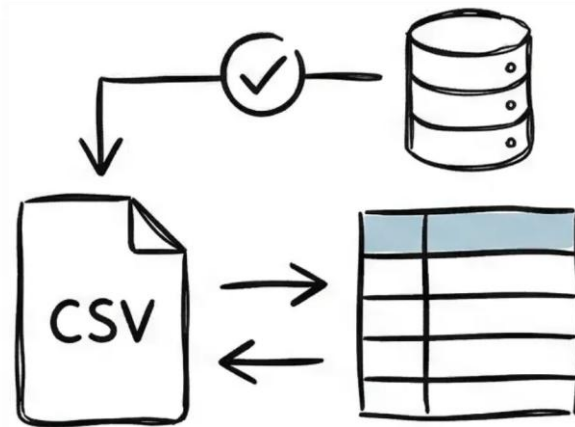
- 원본: 10,000 rows
- DB 적재 후: 10,000 rows
- ☒ 검증 통과!

```
# 원본 데이터 행 수
original_rows = len(df_enriched)
# DB에서 조회한 행 수
db_rows = len(df_from_db)
# 비교 검증
if original_rows == db_rows:
    print("✅ 적재 검증 통과")
```

HANDS_ON 필수 습관

적재할 때마다 반드시
확인하세요

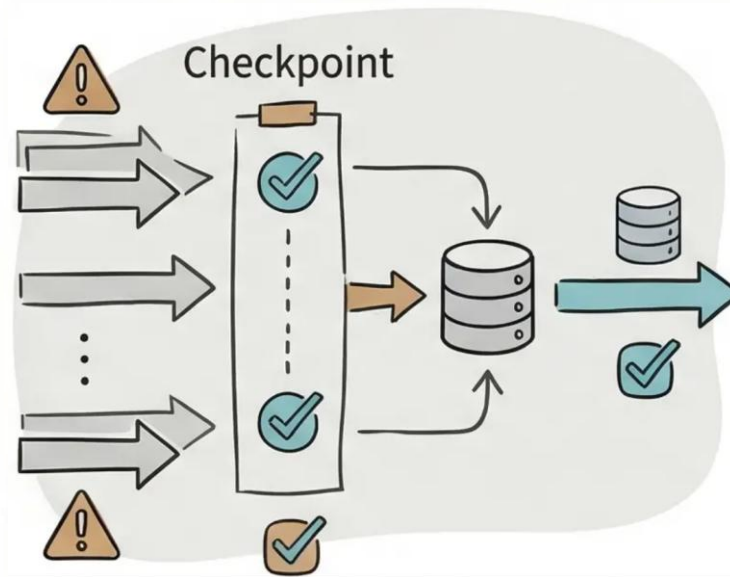
- 원본: 10,000 rows
- DB 적재 후: rows
- ☒ 검증 통과!



중복 적재를 피하는 방법

실습용 접근법

'if_exists='replace''로
테이블 재생성
매번 깨끗하게 시작
교육 환경에서 안전하고
단순함
데이터 손실 위험 없음



실무용 전략

'UPsert' 기능으로
업데이트/삽입
타임스탬프 기반 중복 체크
배치 실행 전 중복 검증

핵심 원칙

중복 방지는 데이터 품질의 기본이며, 운영 환경에서는 신중한 설계가 필요합니다

순차형 ETL 코드 구조 만들기

입문자가 이해하기 쉬운 직렬적 코드 구조를 통해 안정적인 파이프라인 작동을 구현합니다

- 위에서 아래로 실행되는 단순한 구조

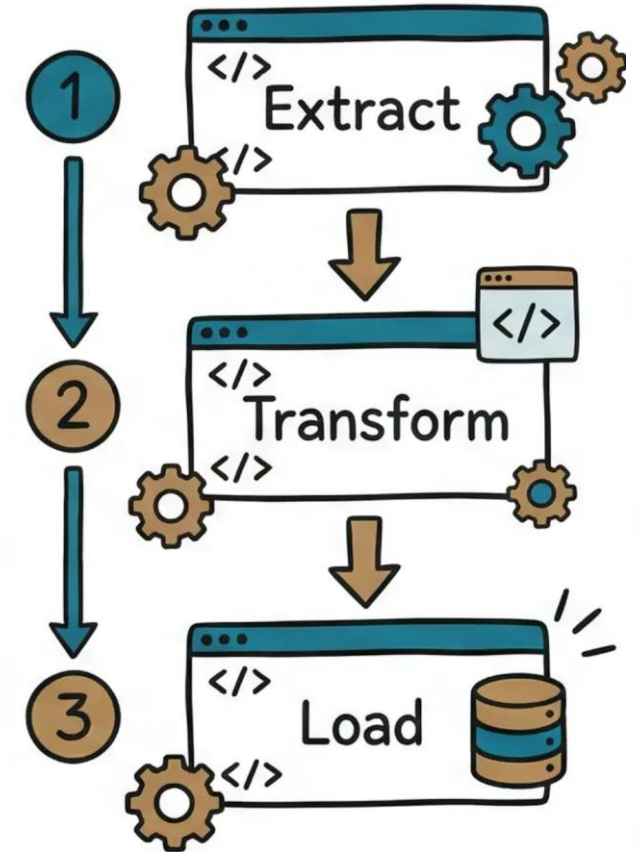
- 복잡한 분기나 병렬 처리 없이 순차 실행
- 각 단계의 결과를 명확하게 확인 가능

- 단계별 명확한 구분과 검증

- Extract → Transform → Load 순서 고정
- 각 단계 완료 후 중간 결과 점검

- 오류 발생 시 추적이 용이한 설계

- 어느 단계에서 문제가 생겼는지 쉽게 파악
- 디버깅과 수정이 간단함



“실무에서는 더 복잡한 구조도 사용하지만, 학습 단계에서는 안정성과 이해도가 우선입니다”

3주차 종합 실습 안내

읽기부터 적재, 조회, 점검, 로깅까지 이어지는 완전한 ETL 파이프라인 구축

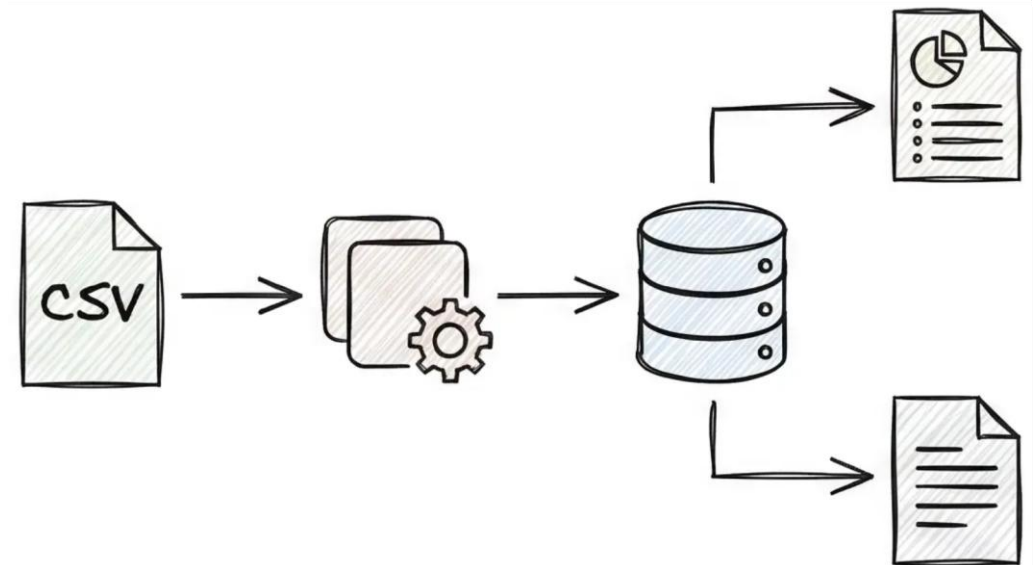
실습 미션 개요

- ai4i_enriched.csv 파일을 SQLite DB로 완전 이관
- 데이터 품질 검증 및 보고서 생성
- 전체 과정 로그 기록 및 추적성 확보

1. **Extract** - 통합 데이터 파일 읽기
2. **Transform** - 최종 품질 점검 수행
3. **Load** - SQLite 테이블로 적재
4. **Validate** - SQL 쿼리로 검증
5. **Report** - 품질 점검 결과 저장
6. **Log** - 실행 과정 기록

Expected Outputs

- manufacturing_pipeline.db
- quality_check_report.csv
- etl_run_log.txt



데이터 품질이란 무엇인가

• 신뢰할 수 있는 데이터의 상태

- 분석과 의사결정에 안전하게 사용할 수 있는 데이터
- 제조 현장의 실제 상황을 정확히 반영하는 정보

• 데이터 품질의 핵심 구성 요소

- 완전성: 필요한 데이터가 모두 있는가
- 정확성: 실제 값과 일치하는가
- 일관성: 동일한 기준으로 기록되었는가
- 적시성: 필요한 시점에 사용 가능한가



품질이 낮은 데이터로 만든 분석 결과는 잘못된 의사결정을 이끌 수 있습니다

결측값과 범위 오류 점검하기

- 자동화된 데이터 품질 검증
 - 적재 후 즉시 실행되는 검증 프로세스
 - 사람의 실수를 줄이는 체계적 점검
- 결측값 점검 방법
 - 컬럼별 결측치 개수와 비율 확인
 - 예상치 않은 결측 패턴 탐지
 - 결측이 허용되는 컬럼과 필수 컬럼 구분
- 값 범위 검증
 - 최소값과 최대값이 예상 범위 내인지 확인 ☒
 - 온도, 압력, 회전수 등 물리적 한계 점검 ☒
 - 음수가 나올 수 없는 값의 검증 ☒



실무 팁

품질 점검 결과를 CSV 리포트로 저장하여
이력 관리와 개선 추적이 가능합니다

중복과 형식 오류 점검하기



- 중복 데이터의 위험성
- 분석 결과 왜곡과 건수 오류 발생
- 통계 신뢰도 저하
- 저장 공간 낭비



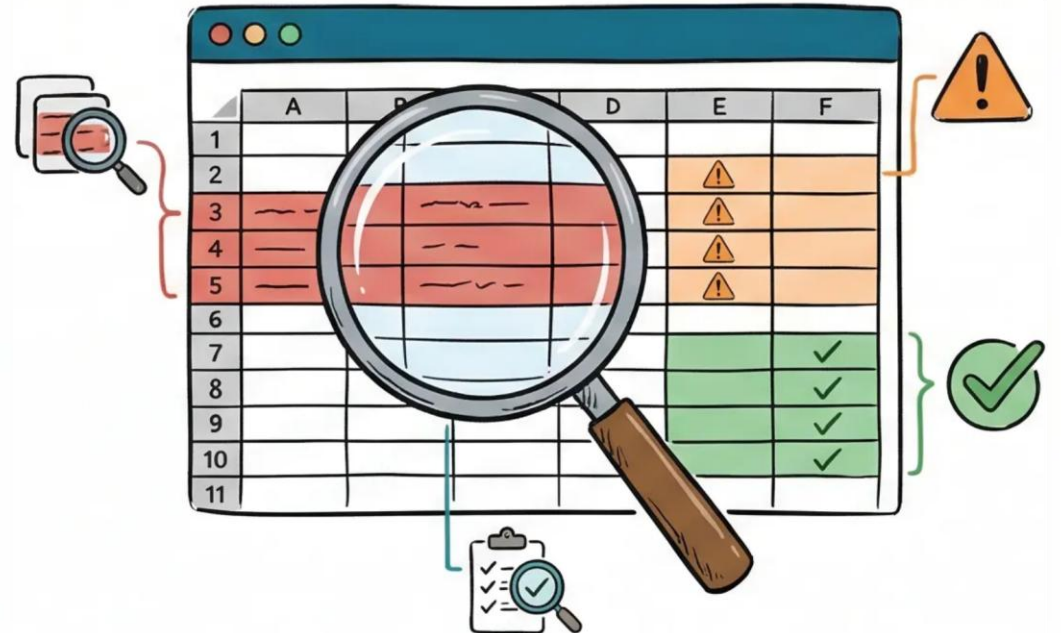
- 형식 오류의 영향
- 시스템 처리 실패 유발
- 데이터 타입 불일치
- 자동화 프로세스 중단



- 점검 방법
- 전체 행 기준 중복 검사
- 핵심 컬럼별 중복 확인
- 데이터 타입 일관성 검증
- 문자열 형식 규칙 점검

⚠ 경고/주의

실무 팁: 중복 제거 전에 반드시 원인을 파악하세요. 의미 있는 중복일 수도 있습니다.



로그를 왜 남겨야 하는가



언제 시작했는지 기록
파이프라인 실행 시간과 소요 시간 추적



무엇을 읽었는지 기록
입력 파일명, 데이터 건수, 소스 정보 저장



몇 건을 처리했는지 기록
처리된 레코드 수, 성공/실패 건수 추적



문제가 있었는지 기록
오류 메시지, 경고 사항, 예외 상황 저장



실무 포인트

로그는 문제가 생겼을 때만 보는 것이 아닙니다.
평상시 운영 현황을 파악하고 성능을 개선하는 중요한 자료입니다.

The screenshot shows a dashboard with a sidebar menu containing 'Dashboard', 'Log', 'Status', and 'Status'. The main content area is divided into three sections:

- Dashboard**: A table with columns 'Lose time', 'Log', 'Status', and 'Status'. It contains five rows of data with timestamps and status indicators (Green, Warning, and Red X).
- Log entries**: A table with columns 'Date' and 'Status'. It contains four rows of data with timestamps and status indicators.
- Data flow tracking**: A diagram showing a flow from a document icon to a play button icon, and then to another document icon.

Lose time	Log	Status	Status
2022-02-09 16:35:59		Green	✓
2022-02-09 16:35:59		Green	✓
2023-02-09 16:35:39		Warning	✗
2022-02-09 16:35:59		Green	✗
2023-02-09 16:35:39		Warning	✗

Date	Status
2023-02-05 15:32:33	
2023-02-05 15:38:33	
2023-02-05 15:35:38	
2023-02-05 15:35:38	

ETL 실행 로그 저장하기

실무 운영법

파이프라인 운영의 투명성과 추적성 확보

실행 시간 기록

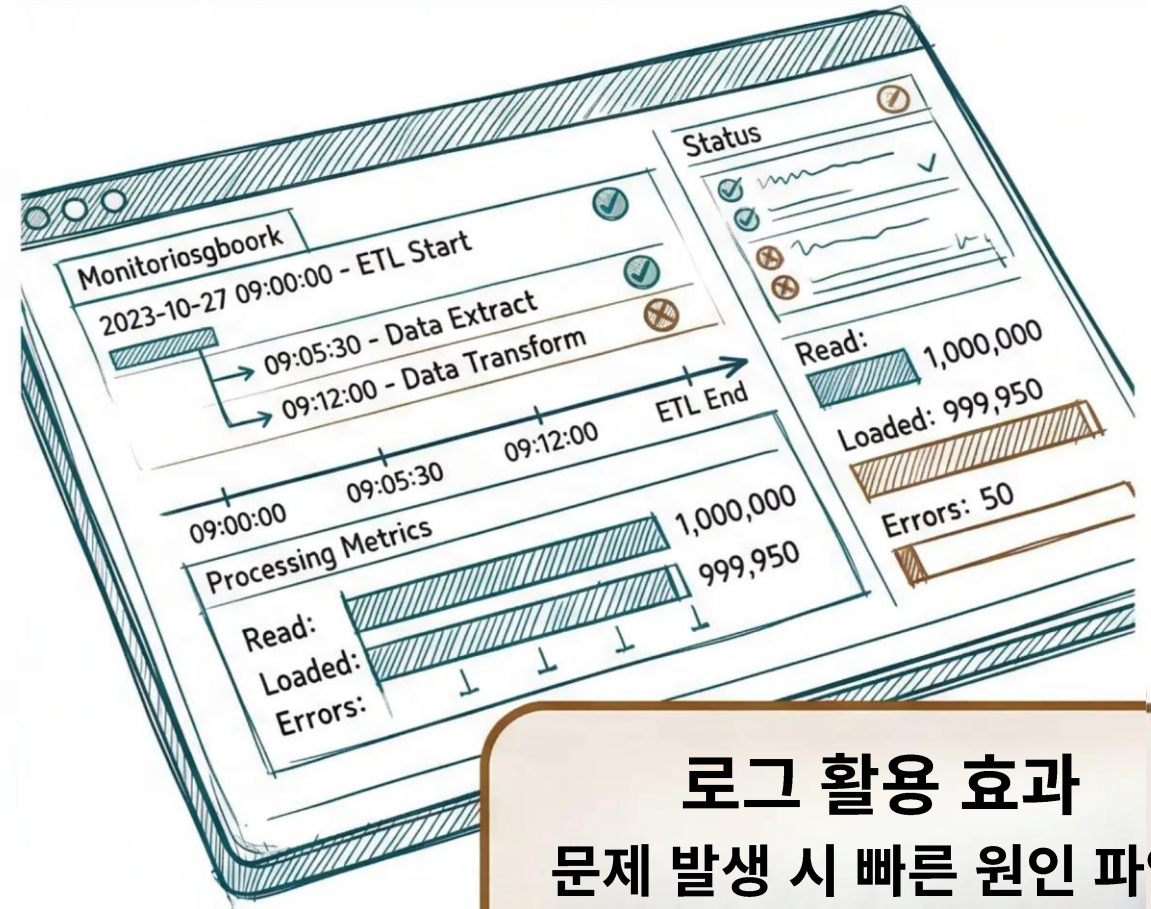
- 시작/종료 시간 자동 기록
- 처리 소요 시간 측정

처리 데이터 현황

- 읽은 데이터 건수
- 성공적으로 적재된 건수
- 오류 발생 건수

상태 및 오류 정보

- 성공/실패 여부
- 오류 메시지 상세 기록
- 단계별 처리 상태



로그 활용 효과

문제 발생 시 빠른 원인 파악
데이터 처리 이력 추적 가능
파이프라인 성능 모니터링

전체 과정 요약과 실무 연결

3주간의 데이터 엔지니어링 여정

- 1주차: 제조 데이터 읽기와 정제의 기초
- 2주차: 다양한 소스 통합과 데이터 결합
- 3주차: SQLite 적재와 품질 검증 완성

현장 적용을 위한 다음 단계

- 실제 제조 데이터로 파이프라인 구축 시작
- 팀 내 데이터 처리 표준화 프로세스 도입
- 정기적인 품질 점검과 모니터링 체계 구축



데이터 엔지니어링의 핵심

읽고 → 정리하고 → 합치고 → 적재하고 → 검증하는 흐름을 마스터했습니다